

PROJECT REPORT

Implement Client Script & UI Policy (Incident)

ServiceNow Incident Management – Naan Mudhalvan Project

Team ID	SWTID-2026-1951
Platform	ServiceNow Personal Developer Instance (PDI)
Project Type	Rule-based ServiceNow configuration
Team Size	5 members

Prepared from the completed Idea, Requirement Analysis, Design, Planning and Development documents supplied for the project.

1. INTRODUCTION

1.1 Project Overview

The project **Implement Client Script & UI Policy (Incident)** improves the ServiceNow Incident form by combining declarative UI Policies with event-based Client Scripts. The solution dynamically controls field behaviour, gives users immediate guidance, validates information before save, and validates inline list edits. The implementation is rule-based and does not use Artificial Intelligence or Machine Learning.

1.2 Purpose

- Reduce incorrect and incomplete Incident data entry.
- Give service desk users immediate feedback while they work on an Incident.
- Use UI Policies for field-state control and Client Scripts for event-specific validation.
- Prevent invalid data at the correct event instead of relying only on post-submission correction.
- Provide a reusable configuration pattern that can be moved using ServiceNow Update Sets.

2. IDEATION PHASE

2.1 Problem Statement

How might we give Incident users immediate guidance and stop invalid data at the correct event using UI Policies and Client Scripts?

The team identified a usability and data-quality gap in the standard Incident workflow: field behaviour was not sufficiently guided as conditions changed, and validation could occur too late.

2.2 Empathy Map Canvas

Perspective	Observed needs / behaviour
Think & Feel	Wants to finish incidents quickly, enter information correctly, and avoid wrong data.
Hear	Needs guidance about required fields and acceptable values.
See	Mandatory/visible/read-only fields, messages, and editable list cells.
Say & Do	Changes fields, submits the form, edits list cells, and corrects entries after messages.
Pain	Invalid data may be entered; guidance may be missing; errors may be found only at save.
Gain	Immediate guidance, correct validation, and more accurate Incident records.

2.3 Brainstorming

Idea	Priority	Purpose
UI Policy Control	High	Control Incident field state dynamically.
UI Policy Action	High	Apply field-level behaviour, especially Urgency.
onChange Client Script	High	React immediately to field changes.
onSubmit Client Script	High	Validate data before saving.
onCellEdit Client Script	Medium	Validate inline list edits.

The prioritized direction was to combine declarative UI Policies with event-based Client Scripts. The project documents explicitly state that the solution is rule-based and does not require AI or Machine Learning.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage	Interaction	System behaviour / goal
Entice	Customer reports an issue	Agent opens the ServiceNow Incident application.
Enter	Start a new Incident	Incident form loads and UI Policy conditions are evaluated.
Engage	Enter details and change Urgency	onChange script and UI Policy Action respond immediately.
Exit	Save/Submit	onSubmit validation allows valid data and blocks invalid data.
Extend	Edit Incident in list	onCellEdit validates inline changes; team reviews and improves rules.

3.2 Solution Requirements

ID	Requirement
FR-1	Control Incident field state dynamically and apply field-level behaviour through UI Policies/UI Policy Actions.
FR-2	React to field changes and provide immediate guidance using an onChange Client Script.
FR-3	Validate Incident data before save and stop invalid submissions using an onSubmit Client Script.
FR-4	Validate inline Incident list edits and block invalid values using an onCellEdit Client Script.
NFR-1 Usability	Provide clear, immediate guidance while the user works.
NFR-2 Security	Follow ServiceNow role-based access control and platform security.
NFR-3 Reliability	Apply rules consistently on field changes, form submission and list edits.
NFR-4 Performance	Respond in the browser without waiting for an additional server round trip.
NFR-5 Availability	Available whenever the ServiceNow instance is available.
NFR-6 Scalability	Apply rule-based logic to Incident records without additional infrastructure.

3.3 Data Flow Diagram

Main flow: Service Desk Agent/Caller → Incident Form/List View → UI Policy & UI Policy Action → Client Scripts (onChange/onSubmit/onCellEdit) → Incident Table. Admin/Developer maintains the policies and scripts through System UI; configuration can be shared using Update Sets.

3.4 Technology Stack

Component	Technology / implementation
User Interface	ServiceNow Next Experience UI / Classic UI in a browser.
Application Logic	JavaScript with ServiceNow client APIs such as g_form and g_user.
Field Behaviour	ServiceNow UI Policies and UI Policy Actions.
Database	ServiceNow Incident [incident] table / platform-managed relational data store.
Cloud Platform	ServiceNow Personal Developer Instance (PDI).
Configuration Storage	ServiceNow Update Sets and platform attachments; GitHub may be used for project files.
External APIs	No separate external API; ServiceNow client APIs are used.
AI/ML	Not used; project is rule-based.
Infrastructure	Cloud-hosted ServiceNow PDI; no local application server required.

4. PROJECT DESIGN

4.1 Problem Solution Fit

The solution fits the identified problem because each user interaction is handled by an appropriate ServiceNow mechanism: UI Policies manage declarative field behaviour, while Client Scripts respond to specific form or list events. This reduces unnecessary user effort and improves data quality without requiring a separate application or machine-learning model.

4.2 Proposed Solution

- **UI Policy Control:** dynamically sets fields as mandatory, read-only or visible.
- **UI Policy Action:** applies field-level behaviour such as the Urgency rule.
- **onChange Client Script:** reacts to a monitored field change and displays guidance.
- **onSubmit Client Script:** validates data before the record is saved.
- **onCellEdit Client Script:** validates inline edits in the Incident list.

4.3 Solution Architecture

The architecture connects the Service Desk user, Incident form, UI Policies, Client Scripts and the ServiceNow Incident table. The solution is rule-based; no AI or Machine Learning component is required.

- Define features, development phases, and solution requirements.
- Provide specifications according to which the solution is defined, managed, and delivered.

Example - Solution Architecture Diagram:

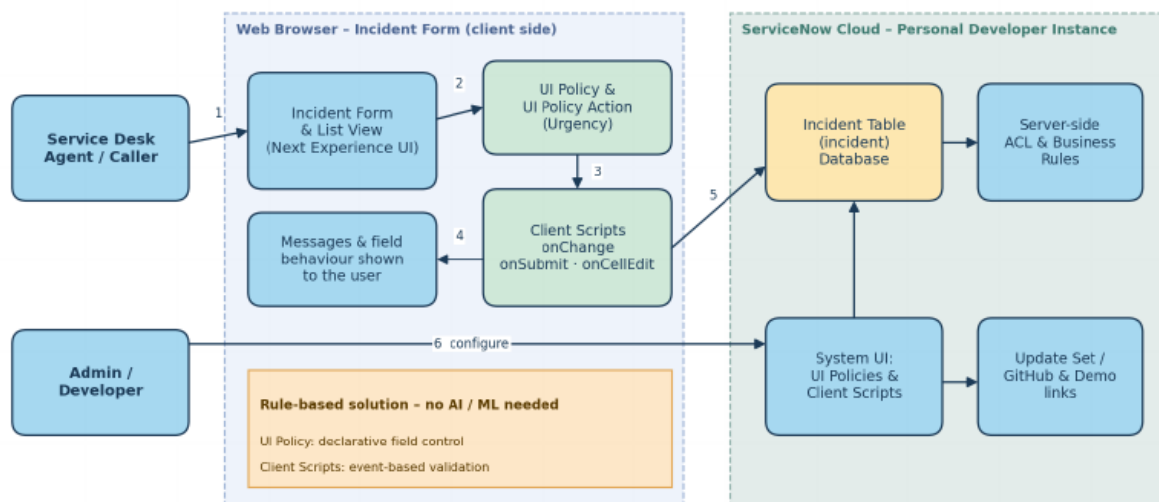


Figure 1: Architecture and data flow of the Client Script & UI Policy solution on the ServiceNow Incident form

1. The agent or caller opens a new or existing Incident in the browser.
2. The UI Policy engine sets field behaviour (mandatory, read-only, visible) as the form loads.
3. When a field such as Urgency changes, the UI Policy Action and onChange Client Script run instantly.
4. The user sees updated fields and on-screen messages without submitting the form.
5. On save, the onSubmit script validates the data; valid data is stored in the Incident table (list edits go through onCellEdit).
6. Admin / developer creates and maintains UI Policies and Client Scripts in System UI and shares the Update Set, demo and GitHub links.

Reference: <https://developer.servicenow.com/>

Figure 1. Architecture and data flow of the Client Script & UI Policy solution on the ServiceNow Incident form.

Flow: the agent or caller opens an Incident → UI Policies apply field behaviour → field changes trigger UI Policy Actions and Client Scripts → the user receives immediate feedback → onSubmit validates before save → valid data is stored in the Incident table; inline edits are checked by onCellEdit.

5.1 Project Planning

Sprint	Major work	Points	Planned dates
Sprint 1	UI Policy, UI Policy Action, onChange Client Script and guidance	10	17 Feb 2025 – 22 Feb 2025
Sprint 2	onSubmit validation, onCellEdit, testing and conclusion	14	24 Feb 2025 – 01 Mar 2025
Total	11 user stories across 7 epics	24	2 sprints

The project planning record gives 24 total story points over two sprints, producing a team velocity of **12 story points per sprint**.

User Stories

USN-1	Create UI Policy on Incident form
USN-2	Add UI Policy Action for Urgency
USN-3	Give immediate Urgency guidance
USN-4	Create onChange Client Script
USN-5	Show instant feedback before saving
USN-6	Create onSubmit save validation
USN-7	Block invalid data and show a message
USN-8	Create onCellEdit Client Script
USN-9	Prevent invalid inline list values
USN-10	Test UI Policies and Client Scripts
USN-11	Document results and links

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

The supplied Functional & Performance Testing record defines functional scenarios FT-01 to FT-05 and performance scenarios PT-01 to PT-03. The target response-time requirement was under 3 seconds for monitored form changes.

Test ID	Scenario	Expected result
FT-01	UI Policy behaviour	Field state changes and reverts according to the condition.
FT-02	UI Policy Action – Urgency	Urgency behaves as configured.
FT-03	onChange Client Script	Guidance appears immediately without form submission.
FT-04	onSubmit validation	Valid data saves; invalid data is blocked with a message.
FT-05	onCellEdit validation	Valid edits are accepted; invalid edits are rejected.
PT-01	Response time	Form response remains under 3 seconds.
PT-02	Rapid field changes	Form remains responsive without errors.
PT-03	Multiple inline edits	Each edit is validated without lag.

AI/ML, Power BI, Tableau and Salesforce-specific model metrics are not applicable because this project is a ServiceNow rule-based configuration.

7. RESULTS

7.1 Output Screenshots

The uploaded project package contains an architecture diagram and testing records, but it does not contain standalone screenshots of the final ServiceNow Incident UI. No UI screenshot has been invented in this report.

Feature	Expected result	Evidence in supplied records
UI Policy	Fields change state based on conditions.	Positive and negative UAT scenarios.
Urgency UI Policy Action	Urgency follows configured behaviour.	Dedicated test scenario.
onChange Script	Immediate guidance appears.	Positive and negative onChange tests.
onSubmit Script	Invalid data cannot be saved.	Valid and invalid save tests.
onCellEdit Script	Invalid inline edits are rejected.	Valid and invalid list-edit tests.

UAT Execution Summary

Section	Total	Not Tested	Fail	Pass
UI Policy on Incident	4	0	0	4
UI Policy Action – Urgency	3	0	0	3
onChange Client Script	4	0	0	4
onSubmit Client Script	4	0	0	4
onCellEdit Client Script	3	0	0	3
Testing the Configuration	2	0	0	2
Total	20	0	0	20

The supplied UAT Execution & Report Submission records 20 total test cases, with 20 passed, 0 failed and 0 not tested.

8. ADVANTAGES & DISADVANTAGES

Advantages	Disadvantages / limitations
Immediate user guidance.	Depends on ServiceNow platform availability.
Reduces invalid or incomplete data entry.	Rules must be maintained as requirements change.
Client-side response reduces extra server round trips for form guidance.	Client-side validation should not be the only security boundary.
Uses native ServiceNow configuration mechanisms.	No AI/ML prediction or learning capability.
Reusable across forms/tables using similar patterns.	Requires ServiceNow access; not a standalone local app.

9. CONCLUSION

The Implement Client Script & UI Policy (Incident) project provides a structured way to improve the ServiceNow Incident data-entry experience. UI Policies handle declarative field behaviour, while onChange, onSubmit and onCellEdit Client Scripts handle event-based guidance and validation. The supplied UAT report records 20/20 test cases passed at the documented testing stage.

10. FUTURE SCOPE

- Extend the same rule pattern to Problem, Change and other ServiceNow forms.
- Add more field-specific validation rules as service-desk requirements evolve.
- Package the configuration in a reusable Update Set for deployment across instances.
- Add ServiceNow reports/dashboards if operational metrics are required.
- Add automated regression tests for a larger number of Incident scenarios.
- Introduce server-side validation/business rules where stronger enforcement is required.

11. APPENDIX

Source Code / Configuration

PROJECT OVERVIEW SCREENSHOT

SkillWallet project overview for “Implement Client Script & UI Policy (Incident)”, showing the project description, team members, milestones, progress and system requirements.

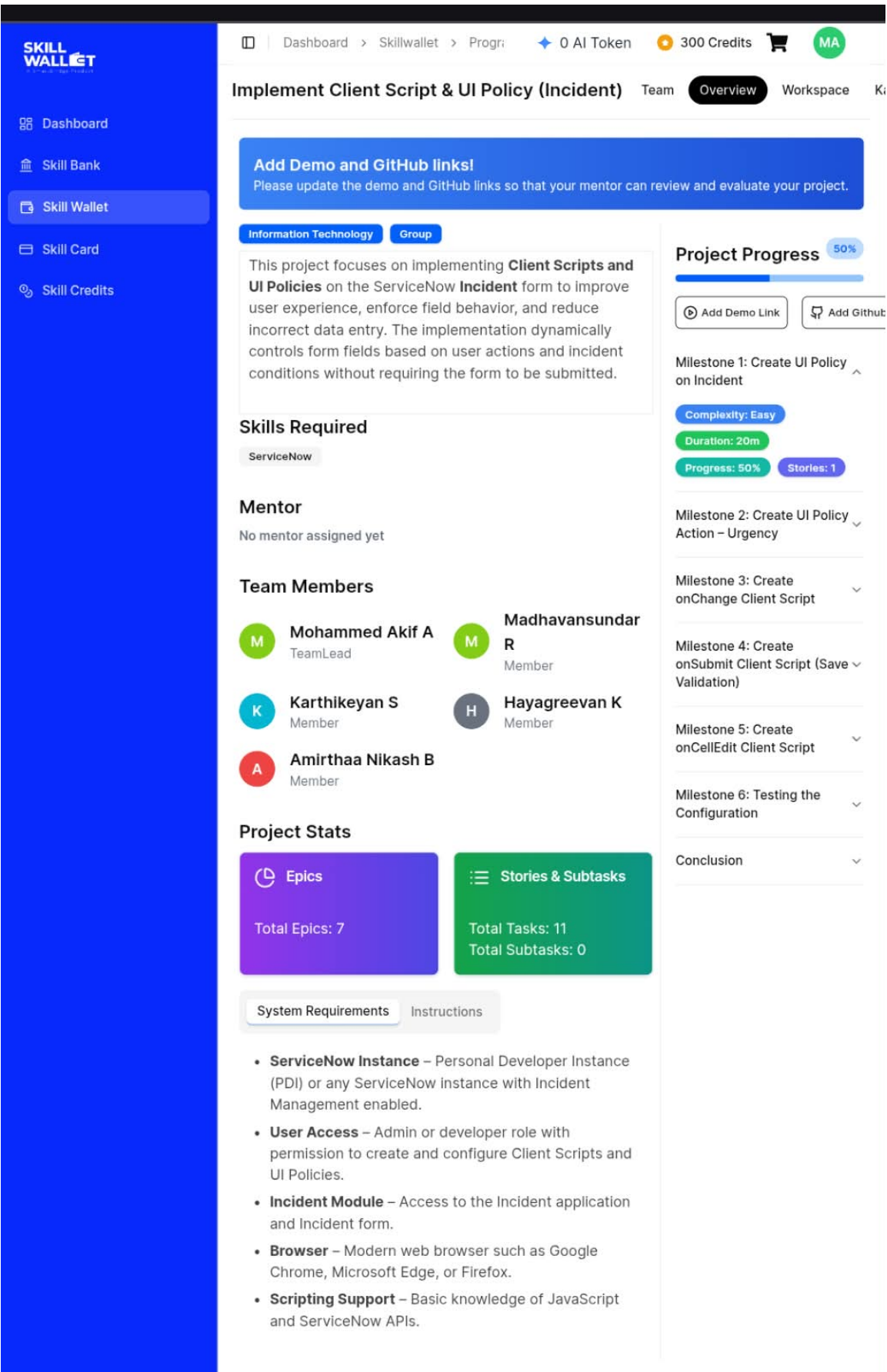


Figure: Supplied SkillWallet project overview screenshot.

The implementation is stored as ServiceNow configuration: UI Policies, UI Policy Actions and Client Scripts. No standalone source-code repository was supplied, so this report does not invent code listings.

Dataset Link

Not applicable. The project does not use a machine-learning dataset.

GitHub & Project Demo Link

No project-specific GitHub or demo URL was supplied in the uploaded documents.

References

- <https://developer.servicenow.com/>
- <https://docs.servicenow.com/>
- <https://c4model.com/>
- <https://www.atlassian.com/agile/project-management>
- <https://www.atlassian.com/agile/tutorials/burndown-charts>

Team Members

Team Member	Assigned contribution
Mohammed Akif A	UI Policy; testing; conclusion/documentation
Madhavansundar R	UI Policy Action – Urgency; user guidance
Karthikeyan S	onChange Client Script; instant feedback
Hayagreevan K	onSubmit Client Script; save validation
Amirthaa Nikash B	onCellEdit Client Script; inline validation